

Telegram Smart Task Reminder Bot Full Product Requirements Document (PRD)

This document describes the full MVP specification for a personal productivity Telegram bot that integrates with Notion and Google Calendar. The system proactively suggests tasks during free time windows and allows the user to manage tasks directly from Telegram.

1. Product Vision

The product acts as a lightweight productivity companion designed to reduce task blindness and executive-function friction. The bot continuously monitors: - tasks stored in Notion - upcoming calendar events - available free-time windows It then proactively suggests relevant tasks through Telegram notifications. The system is intentionally non-AI and deterministic in order to: - remain predictable - stay lightweight - reduce infrastructure costs - simplify implementation

2. Problem Statement

The user stores tasks in Notion because it is fast and comfortable for capturing information. However: - Notion is not checked frequently enough - Tasks become “invisible” after being written down - Small free time slots are wasted - The user often believes there is nothing left to do - Calendar availability and tasks are disconnected The product solves the activation problem rather than the organization problem.

3. Target Audience

Primary audience: - Single personal user (V1) Future audience: - Multi-user productivity SaaS users - Users with ADHD/executive-function challenges - Heavy Notion users - Calendar-driven professionals

4. Core Integrations

The system integrates with: 1. Telegram - Primary user interface - Notifications - Interactive task management 2. Notion - Source of truth for tasks - Task status updates - Task metadata storage 3. Google Calendar - Free-time detection - Event awareness - Scheduling context

5. Notion Database Structure

The bot reads from a dedicated Notion task database. Required properties: - Name (title) - Text / Description (rich text) - Due Date (date) - Status (select) - Estimated Time (number in minutes) Supported statuses: - Inbox - In Progress - Done Tasks marked as Done must never be shown in reminders.

6. Free Time Detection Rules

A free time slot is considered valid when: - The gap between two calendar events is greater than 30 minutes - Current time is before 20:00 - Weekdays: - reminders only after 08:00 - Weekends: - reminders are allowed throughout the day The scheduler continuously scans the calendar to identify valid gaps.

7. Task Suggestion Logic

Task selection is deterministic and rule-based. Selection priority: 1. Tasks closest to due date 2. Tasks fitting the available free-time duration 3. Tasks not marked as Done Additional behavior: - Tasks due tomorrow are highlighted - Short free slots prioritize shorter tasks - Long free slots may include larger tasks Examples: - 20-minute slot → quick tasks - 90-minute slot → deep tasks

8. Reminder Experience

Example reminder: "You have 45 free minutes before your next event. Suggested tasks: - Renew passport - Reply to Dan - Gym" Each task appears as an inline Telegram button.

9. Task Detail Screen

When the user clicks a task button, the bot displays: - Task title - Due date - Description - Current status - Estimated duration Available actions: - ☐ Done - ☐ Remind Later - ☐ Doing... Behavior: - Done → updates Notion status to Done - Doing... → updates status to In Progress - Remind Later → no status change

10. Persistent Telegram Commands

The Telegram bot includes persistent shortcut buttons: - What should I do now? - Show me quick tasks - What can I finish in 20 minutes? These commands trigger filtered task suggestions.

11. Reminder Frequency

The reminder system should be proactive but not spammy. Recommended rules: - Maximum one proactive reminder every 2–3 hours - Avoid repeatedly suggesting the same task too frequently - Skip reminders during busy periods

12. Technical Architecture

Recommended stack: Backend: - Python + FastAPI Telegram: - python-telegram-bot Database: - SQLite for MVP - PostgreSQL for future public release Infrastructure: - Background scheduler/worker - Webhook-based Telegram

communication - REST integrations with Notion and Google APIs

13. Internal Scheduler Flow

Every few minutes: 1. Read current calendar events 2. Detect upcoming free slot 3. Fetch active tasks from Notion 4. Filter tasks by duration and status 5. Rank tasks by urgency 6. Send Telegram reminder if conditions are met

14. Non-Goals for MVP

The following are intentionally excluded from V1: - AI recommendations - Automatic calendar scheduling - Natural language understanding - Voice input - Recurring tasks - Team collaboration - Apple Calendar support - Analytics dashboards

15. Future Roadmap

Future improvements may include: Productivity Features: - recurring tasks - focus sessions - smart snoozing - work modes Context Awareness: - energy-based task filtering - location-aware tasks - context tags Public SaaS Features: - multi-user architecture - OAuth onboarding - subscriptions - cloud deployment

16. Success Criteria

The MVP is successful if: - The user consistently completes more tasks - Small free time slots become productive - Telegram becomes the primary task interaction layer - The user checks Notion less frequently but misses fewer tasks
